Proyecto 3: Pochoco SoC, de RTL a GDSII

Arquitectura de Computadores, Universidad de los Andes, 2026-2

Santiago Arnolds Lucas Nestler Matias Veto

4 de noviembre de 2026

Índice

3.1 Del RTL de FPGA al RTL de ASIC 3
El top del macro: `asic_top` 3
Método 3
Construcciones encontradas 3
Lo que mostró LibreLane 4
Problemas que no generaron ningún mensaje 4
Verificación de los cambios 5
3.2 Memoria y programa del juego 6
Palabras que usa el juego 6
El assembler genera la ROM 6
Las dos estrategias 6
Comparación 7
Elección: estrategia 1 (ROM) 7
3.3 Síntesis 8
Cómo se conservó la jerarquía 8
Área por módulo 8
Las 5 celdas más usadas 8
Comparación con lo esperado 9
Síntesis vs. layout final 9
3.4 Timing 10
Corners 10
Barrido de `CLOCK_PERIOD` 10
Ruta crítica de la corrida final 11
Optimización de la ruta crítica 11
Constantes de `game.s` para el período T 12
3.5 Floorplan, alimentación y placement 13
Floorplan 13
Alimentación (PDN) 13
Terminales del macro 13
Barrido de utilización 14
Costo 16
3.6 Signoff y layout 17
Resultado del signoff 17
Violación de antena durante el desarrollo 17
Violaciones fuera del criterio: max slew, cap y fanout 19
Layout anotado 20
3.7 Simulación funcional post-síntesis 22
Programa con los tiempos acortados 22
Testbench 22
Resultado en RTL 22
Resultado en el netlist 22
Qué demuestra y qué no 23
Cómo correrlo 23

3.1 Del RTL de FPGA al RTL de ASIC

El top del macro: `asic_top`

`rtl/asic_top.v` instancia el SoC (`pochoco_soc u_pochoco`) y no contiene nada propio de la Go Board. Sus puertos son terminales lógicos del macro, no pads de I/O:

Puerto	Dir.	Función
`clk_i`	in	reloj (10 MHz en la corrida final, `CLOCK_PERIOD` = 100 ns)
`rst_ni`	in	reset asíncrono, activo bajo (no existía en el Proyecto 2)
`btn_i[3:0]`	in	botones
`led_o[3:0]`	out	LEDs
`seg1_o[6:0]`, `seg2_o[6:0]`	out	displays de 7 segmentos, `{G..A}`, **activo alto**
`spi_sclk_i`, `spi_mosi_i`, `spi_cs_ni`, `spi_miso_o`	in/out	esclavo SPI

La estrategia de memoria (sección 3.2) se elige con un parámetro: ROM por defecto, o flip-flops si se define `POCHOCO_MEM_FF`. `game_top.v` (el top de la FPGA) sigue en el repositorio, pero esta rama ya no apunta a la Go Board: la versión para la placa es el tag `p2-entrega`.

Método

Para que cada construcción dependiente de la FPGA quedara respaldada por un mensaje del flujo, se corrió LibreLane **antes de arreglar nada** y se guardaron las corridas en `flow/asic_top/runs/`:

Corrida	RTL	Qué mostró
`00_rtl_p2_sin_cambios`	RTL del P2 + `asic_top`	El flujo se detiene en la etapa 3 (`Checker.LintErrors`) por `SB_RAM40_4K`; tapa todo lo demás
`01_regfile_ff`	`00` + register file de flip-flops	Lint y síntesis pasan: aparecen los mensajes de los puntos 2–5
`02_reset_externo`	`01` + reset externo y sin `initial`	Confirma que los mensajes de los puntos 4 y 5 desaparecen

Construcciones encontradas

#	Construcción	Archivo	Mensaje de LibreLane (log:línea, en `flow/asic_top/runs/`)	Reemplazo
1	Primitiva `SB_RAM40_4K` (BRAM de la iCE40)	`espino_register_file.v`	`00_rtl_p2_sin_cambios/01-verilator-lint/verilator-lint.log:1` y `:13`: `%Error-MODMISSING ... 'SB_RAM40_4K'`	15 × 32 flip-flops con lectura sincrónica y reset a 0
2	`initial $readmemh`	`pochoco_ram.v`	`01_regfile_ff/06-yosys-synthesis/yosys-synthesis.log:1093` (`meminit`) y `:4102–4106` (`DFFLEGALIZE`); **sin warning**	Programa dentro del circuito: ROM generada por el assembler o valores de reset (3.2)
3	Inferencia de BRAM de 512 palabras	`pochoco_ram.v`	`01_regfile_ff/.../yosys-synthesis.log:1401–1406` (`MEMORY_MAP`: 512 × 32 FF) y `:4558` (2,31 mm²)	Memoria dimensionada al uso real (99 instrucciones, 0 datos)
4	Reset de encendido por valor inicial (`reg [12:0] por_cnt = 13'b0`)	`pochoco_soc.v`	`01_regfile_ff/01-verilator-lint/verilator-lint.log:7` (`PROCASSINIT`) y `:1` (`rst_ni` sin usar); `.../yosys-synthesis.log:304–305` (`PROC_INIT`)	Puerto `rst_ni` + sincronizador de 2 FF

#	Construcción	Archivo	Mensaje de LibreLane (log:línea, en `flow/asic_top/runs/`)	Reemplazo
5	`initial` y registros sin reset	`debouncer.v`	`01_regfile_ff/.../yosys-synthesis.log:299–303` (`PROC_INIT`); **sin warning**	Reset asíncrono a 0
6	Reloj de 25 MHz supuesto	`debouncer.v`, `game.s`	Sin mensaje	`TICK_BITS` 18 ⟶ 17; constantes de `game.s` para 10 MHz
7	Inversión `~seg` (displays activo bajo de la placa)	`pochoco_soc.v`	Sin mensaje	Salidas activo alto; la inversión pasa a `game_top`
8	Botones asíncronos sin sincronizador	`debouncer.v`	Sin mensaje	2 FF de sincronización por botón

El extracto de cada mensaje está en `docs/evidencia.md`.

Lo que mostró LibreLane

Error explícito (1). Solo `SB_RAM40_4K` detuvo el flujo. Es un bloque físico de la iCE40 instanciado a mano en el register file (dos copias, para tener dos puertos de lectura). La biblioteca `gf180mcu_fd_sc_mcu7t5v0` solo tiene celdas estándar y el proyecto no usa macros SRAM, así que Verilator no encuentra el módulo. Se reescribió `espino_register_file` con flip-flops, sin cambiar el nombre del módulo, los puertos ni la instancia `u_rf`. Se mantuvo la **lectura sincrónica** de la BRAM, porque el core depende de esa latencia de un ciclo (FSM `ops_valid_q` de `espino_id_stage.v`), y se agregó **reset a 0** para reproducir el `INIT_0 = 0` de la BRAM. Costo: 544 flip-flops. En la corrida final, el register file es el 45 % del área lógica (82.990 µm², `docs/img/bloques.csv`).

Valores iniciales descartados en silencio (2, 4, 5). El hallazgo central de esta sección es que `$readmemh`, los bloques `initial` y el valor inicial de `por_cnt` **no producen ningún error**. Yosys los convierte en atributos `init` (`PROC_INIT`, `meminit`), y luego `DFFLEGALIZE` mapea esos flip-flops a celdas `dffq_1`, que no tienen valor inicial (`yosys-synthesis.log:4106`: 16.488 FF). El valor desaparece del netlist sin aviso. El único indicio fue el warning `PROCASSINIT` de Verilator sobre `por_cnt`. En la FPGA esos valores existen porque el bitstream inicializa cada FF y cada BRAM al configurarse. En silicio los FF arrancan en un valor indeterminado, lo que implica:

- **(2)** El programa no estaría en la memoria: el SoC ejecutaría basura.
- **(4)** `por_cnt` podría arrancar con el bit 12 en 1, y el SoC nunca pasaría por reset. En simulación del netlist queda en X y el reset no se suelta nunca.
- **(5)** El debouncer podría reportar botones falsos durante los primeros muestreos.

Reemplazos:

- El programa pasa a ser parte del circuito (3.2).
- `asic_top` tiene un puerto `rst_ni` que entra a un **sincronizador de 2 flip-flops** en `pochoco_soc`: aserción asíncrona (no necesita reloj) y liberación sincrónica (todos los FF salen de reset en el mismo flanco, sin violaciones de recovery/removal).
- `por_cnt` se eliminó: sus 4.096 ciclos de espera eran para que las BRAM de la iCE40 se estabilizaran.
- `debounce_bank` tiene reset asíncrono.

En `02_reset_externo` ya no aparecen ni `PROCASSINIT` ni las reglas `PROC_INIT`. Además, en la simulación post-síntesis (3.7) todos los FF arrancan en X y el juego funciona: eso muestra que el SoC ya no depende de valores iniciales.

Área desbordada (3). Sin BRAM, `MEMORY_MAP` convierte la RAM de 512 × 32 en 16.384 flip-flops con sus multiplexores. El diseño llega a 67.817 celdas y 2,31 mm² de área de celdas, el 47,8 % secuencial (`yosys-synthesis.log:4558–4559`). No es un error, pero casi toda esa área es memoria que el juego no usa. Con la memoria dimensionada al uso real (99 instrucciones y 0 datos, `scripts/contar_palabras.py`), la corrida final tiene 196.097 µm² de celdas, unas 12 veces menos.

Problemas que no generaron ningún mensaje

El flujo no tiene cómo detectarlos, porque son correctos para la síntesis y el STA. Son supuestos sobre el entorno de la Go Board:

- **(6) Reloj.** Un contador de 18 bits o la constante 75.000.000 son igual de válidos para Yosys. Pero el ASIC corre a 10 MHz, no a los 25 MHz del oscilador de la placa, así que todo lo que mide tiempo contando ciclos queda 2,5 veces más

lento: los 3 s de espera pasarían a 7,5 s, cada décima mostrada valdría 0,25 s y el debouncer muestrearía cada 26 ms. Se cambió `TICK_BITS` a 17 (2^17 / 10 MHz ≈ 13,1 ms) y se recalcularon las constantes de `game.s` (3.4).

- **(7) Polaridad de los displays.** El activo bajo es una propiedad de los displays de la Go Board, no del SoC. Un macro no sabe qué se le va a conectar, así que dejar la inversión adentro fijaría en silicio una convención de la placa. `asic_top` entrega los segmentos activo alto, y la inversión quedó solo en `game_top`.
- **(8) Botones asíncronos.** Un botón cambia en cualquier momento respecto del reloj. En el P2, los botones entraban directo a los flip-flops de muestreo. Si uno cambia dentro de la ventana de setup/hold, ese FF puede quedar metaestable. El STA no puede verificarlo, porque la entrada no tiene relación temporal con `clk_i`. En la FPGA nunca falló, pero en el ASIC nada lo garantiza. Se agregaron 2 FF en cadena por botón (8 FF en total), igual a lo que ya tenía el esclavo SPI. La latencia extra (2 ciclos) queda absorbida por el período de muestreo del debouncer.

Verificación de los cambios

- `sim/tb_register_file.v`: reset, x0, lectura sincrónica, read-before-write y 2.000 accesos aleatorios contra un modelo de referencia.
- `sim/tb_game_smoke.v`: con el register file nuevo da la misma salida y los mismos tiempos que con el modelo `SB_RAM40_4K`. Con el reset externo, los eventos se adelantan exactamente 4.090 ciclos (el reset de 4.096 ciclos pasa a ~6).
- `sim/tb_pochoco_periph.v` pasa.
- Lint de Verilator en la corrida final (`CP100_U40_final/01-verilator-lint/verilator-lint.log`): 0 errores y 14 warnings, todos de señales o parámetros sin usar (`UNUSEDSIGNAL`, `UNUSEDPARAM`). Por ejemplo: bits altos de direcciones que la memoria no decodifica, `shamt` y los parámetros de shift de la ALU (los shifts están deshabilitados desde el P2) o `illegal_insn`. Ninguno es una construcción dependiente de la FPGA.

3.2 Memoria y programa del juego

En la FPGA, el programa y los datos vivían en una RAM unificada de 512 palabras (`pochoco_ram`), inferida como BRAM e inicializada con `$readmemh`. En el ASIC no hay BRAM ni carga de archivo al encender (puntos 2 y 3 de 3.1), así que la memoria se reemplazó por dos alternativas y se eligió una comparando corridas completas. Detalle y comandos en `docs/memoria.md`.

Palabras que usa el juego

	Palabras	**Cómo se obtuvo**
Instrucciones	**99** (396 bytes, `0x000–0x188`)	`scripts/contar_palabras.py sw/game.s`: palabras que emite el assembler. Es la imagen completa del programa, se ejecuten o no todas las rutas
Datos	**0**	Mismo script: los 19 loads/stores de `game.s` usan como base `s0`, que solo se escribe con `lui s0, 0x80000`. Todos van a MMIO (`addr[31] = 1`) y ninguno a RAM. El assembler no acepta `.data`/`.word`, así que tampoco hay datos iniciales

El conteo es estático y se confirmó en simulación: `sim/tb_game_fixed_stimuli.v` cuenta durante el juego los accesos de datos a RAM (**0**) y la dirección de instrucción más alta pedida (`0x18c`, la palabra 99). Esa última es el *prefetch* de PC+4 que hace el fetch antes de que el salto final redirija: lee 0, que el core ejecuta como no-op.

Las dos cifras quedan bajo los límites recomendados (256 palabras de instrucciones, 64 de datos), así que no hubo que justificar una excepción.

El assembler genera la ROM

El assembler del Proyecto 2 (`sw/assembler.py`) ganó la opción `--rom`, que además del `.hex` escribe el módulo Verilog de la ROM:

```
python3 -B sw/assembler.py sw/game.s sw/game.hex --rom rtl/game_rom.v
```

`game_rom` es combinacional: `addr_i[7:0]` es la dirección de **palabra** (256 palabras como máximo), un `always @*` con un `case` que tiene una línea por instrucción, y `default = 0` (no-op). `rtl/game_rom.v` es un archivo generado, no se edita a mano. Si el programa pasa de 256 palabras, el assembler da error en vez de truncar.

Validación: `sw/test_assembler.py` (`TestRomVerilog`) prueba el formato, el límite de 256 palabras, la línea de comandos y que la ROM de `game.s` tenga exactamente las palabras de `game.hex` en las mismas direcciones. Además, la salida coincide con la de `scripts/rom_provisoria.py` (un script independiente que lee el `.hex`) salvo los comentarios de cabecera.

Las dos estrategias

`pochoco_soc` tiene el parámetro `MemStrategy`, y `asic_top` usa `"rom"` salvo que se defina `POCHOCO_MEM_FF`. Las dos memorias tienen la misma interfaz que la RAM original (dos puertos con lectura sincrónica), así que el core no cambia.

	Estrategia 1: `pochoco_mem_rom`	**Estrategia 2: `pochoco_mem_ff`**
Instrucciones	`game_rom` combinacional + registro de salida de 32 FF	107 palabras de FF (99 de programa + 8 de datos), cargadas en el reset con la imagen de `game_rom`
Datos	RAM de 8 palabras de FF, sin reset, con *byte enables*	En la misma memoria, después del código
Arquitectura	Harvard: el puerto de datos no lee la ROM	Unificada, como la RAM del Proyecto 2
Flujo	`flow/asic_top`	`flow/asic_top_ff` (`VERILOG_DEFINES: POCHOCO_MEM_FF`)

Por qué 8 palabras de datos si el juego usa 0. Se deja una RAM chica, igual en las dos estrategias, para que el SoC pueda correr programas con un poco de pila (guardar `ra` y algunos registros en llamadas anidadas). La comparación no se ve afectada porque las dos la tienen.

Cómo se carga la memoria FF. Cada palabra de programa se instancia con una dirección constante de `game_rom` como valor de reset. Yosys reduce esas instancias a constantes: no queda ninguna ROM en el netlist, y cada bit pasa a ser un FF con reset a 0 o con *set* a 1. En una síntesis genérica de comprobación, los FF de tipo *set* son **1.096**, exactamente

la cantidad de bits en 1 de `game.hex`. Que sea el mismo número que los FF de la corrida con ROM (tabla de abajo) es coincidencia: son conteos distintos.

Mismo comportamiento. Con los mismos estímulos, el juego da exactamente los mismos eventos en los mismos ciclos con las dos memorias (`sim/run_fixed_stimuli.sh rtl rom` y `rtl ff`, comparados con `compara`: **IGUALES**).

Comparación

Corridas `flow/asic_top/runs/CP100_U40_rom` y `flow/asic_top_ff/runs/CP100_U40_ff`, flujo completo con T = 100 ns y `FP_CORE_UTIL` = 40 %. Las dos usan el mismo RTL y la misma configuración; solo cambia `POCHOCO_MEM_FF`. Los números salen de `results/memoria.csv` (`scripts/extract_metrics.py`).

	Estrategia 1 (ROM)	**Estrategia 2 (FF)**	**FF / ROM**
Área de celdas (µm²)	**211.393**	816.279	3,9×
Celdas estándar	9.638	32.356	3,4×
Flip-flops	**1.096**	4.265	3,9×
Utilización final	51,6 %	53,9 %	
Área del die (µm²)	**440.736** (0,44 mm²)	1.571.020 (1,57 mm²)	3,6×
Setup WS típico / lento (ns)	64,96 / 33,30	56,34 / 16,09	
Hold WS, peor corner (ns)	0,152	0,122	
Frecuencia máxima reg→reg (MHz)	**15,0**	11,9	
Violaciones de antena (redes)	0	1	
¿Cierra a 100 ns / 40 %?	**sí**	**no** (antena)	

La frecuencia máxima es 1000 / (T − WS de setup reg→reg), tomando el peor entre el corner típico (`nom_tt_025C_5v00`) y el lento (`max_ss_125C_4v50`). Es una estimación desde una sola corrida; el período mínimo real sale del barrido de 3.4.

Estas dos corridas son anteriores a la optimización del LSU de 3.4 y a la configuración final (jerarquía conservada, pines, DRC de KLayout). Sirven para comparar las estrategias entre sí con todo lo demás igual. La corrida final `CP100_U40_final` usa la ROM con el RTL final: 196.097 µm² de celdas, 9.651 celdas y 0,4387 mm² de die.

Por qué la FF no cierra. Setup y hold pasan en todos los corners; lo que falla es la antena. Después del ruteo global hay 125 redes (132 pines) con violación (`40-openroad-checkantennas/reports/antenna_summary.rpt`). El repair las deja en 0, pero el ruteo detallado vuelve a crear una: `net4125` ⟶ `fanout4124/I`, Metal2, razón 472,61 / 400 (`46-openroad-checkantennas-1/reports/antenna_summary.rpt`). Se podría corregir con más iteraciones de repair o un diodo, pero no cambia la decisión.

Elección: estrategia 1 (ROM)

- **Área y costo.** La ROM es unas 4 veces más chica en celdas, FF y die, y por lo tanto en costo (3.5).
- **Timing.** Tiene más margen: 33,3 ns de setup en el corner lento contra 16,1 ns.
- **Cierre.** Cierra sin intervención; la FF deja una violación de antena.

La diferencia viene de cómo se guarda un programa que no cambia. La memoria FF paga 32 FF por palabra (la mayoría nunca se escribe) más un mux de lectura de 107 entradas en cada uno de los dos puertos. En la ROM, esas mismas palabras son lógica combinacional que Yosys minimiza aprovechando bits repetidos y palabras parecidas, y los 1.096 FF que quedan son casi todos del core y los periféricos. Lo único que la FF ofrece de más es la arquitectura unificada (un programa podría modificarse a sí mismo o usar todo el espacio para datos), y el juego no lo usa.

El costo de la ROM es que el programa queda fijo en el silicio: cambiar `game.s` (por ejemplo, las constantes de tiempo de 3.4) obliga a regenerar `game_rom.v` y repetir el flujo completo.

3.3 Síntesis

Corrida final `flow/asic_top/runs/CP100_U40_final` (T = 100 ns, U = 40 %, memoria ROM). Fuente de las tablas: `06-yosys-synthesis/reports/stat.rpt` de esa corrida, resumido en `results/sintesis.md` con `python3 scripts/analisis_sintesis.py flow/asic_top/runs/CP100_U40_final -o results/sintesis.md`.

Cómo se conservó la jerarquía

LibreLane aplana el diseño completo por defecto (`SYNTH_HIERARCHY_MODE = flatten`), y con eso el register file, la ALU y el contador de ciclos desaparecen como módulos del netlist. Para poder medirlos, en `flow/asic_top/config.json` se declararon esos tres módulos en `SYNTH_KEEP_HIERARCHY_MODULES`. Yosys los sintetiza por separado (atributo `keep_hierarchy`) y aplana todo lo demás en `asic_top`. Las instancias son:

Módulo	Instancia
`espino_register_file`	`u_pochoco.u_core.u_rf`
`espino_alu`	`u_pochoco.u_core.u_id.u_alu`
`pochoco_cycle_counter`	`u_pochoco.u_periph.u_cycle_counter`

El contador de ciclos estaba en el Proyecto 2 como un `always` dentro de `pochoco_periph`. Se extrajo a su propio módulo (`rtl/pochoco_cycle_counter.v`), sin cambiar su comportamiento, para poder identificarlo.

Método de medición. `06-yosys-synthesis/reports/stat.rpt` trae una sección por módulo con la cantidad y el área **locales** (sin submódulos). La suma de las secciones es el área total del top (`Chip area for top module`). El script `scripts/analisis_sintesis.py` lee ese archivo y arma las tablas de abajo. En el layout final OpenROAD aplana el netlist, pero cada celda conserva el nombre de su instancia como prefijo (por ejemplo `u_pochoco.u_periph.u_cycle_counter/_130_`), lo que permite ubicarlas en el layout (3.6).

Costo de conservar la jerarquía. Comparando `CP100_U40_prueba` (aplanado) con `CP100_U40_jerarquia`, dos corridas iguales salvo por esta opción (anteriores a la exclusión de las celdas `dly*` de la corrida final): el área de celdas sube un 0,2 % (210.867 → 211.325 µm²) y el slack de setup en el corner lento baja 2,5 ns (38,98 → 36,46 ns), porque Yosys no puede optimizar a través de los bordes de esos tres módulos. A 100 ns es un costo despreciable.

Área por módulo

Área de celdas después de la síntesis (antes de place & route: sin los buffers de repair, el árbol de reloj, ni las celdas de tap y relleno).

Módulo	Área (µm²)	% del total	Celdas	Flip-flops
Resto del SoC (`asic_top`, aplanado)	77.616	47,4 %	3.090	522
`espino_register_file`	71.588	43,7 %	1.804	544
`espino_alu`	10.006	6,1 %	580	0
`pochoco_cycle_counter`	4.632	**2,8 %**	162	32
Total	**163.841**	100 %	**5.636**	**1.098**

- **El módulo con más área es el register file** (43,7 %). El "resto del SoC" es algo más grande, pero no es un módulo: es todo lo que se aplanó en el top (memoria, decoder, LSU, IF, periféricos, SPI, debouncer).
- **El contador de ciclos es el 2,83 % del área de celdas** (4.632 µm²): 32 flip-flops con reset, un incrementador de 32 bits y el mux de carga para la escritura por MMIO.

Las 5 celdas más usadas

Celda	Total	Register file	ALU	Contador	Resto	Qué la explica
`nand2_1`	976	142	82	39	713	Lógica combinacional general: decoder, mux de operandos, decodificación de direcciones y periféricos
`dffrnq_1`	776	544	0	32	200	Flip-flop con reset asíncrono. 544 del register file (15 registros × 32 bits + 2 × 32 de los registros de lectura sincrónica), 32 del contador y el resto PC, periféricos, SPI y debouncer
`aoi22_1`	615	384	9	0	222	Árboles de mux de lectura del register file (dos puertos de lectura de 16 a 1 de 32 bits)
`mux2_2`	554	480	0	2	72	Habilitación de escritura: la biblioteca no tiene flip-flops con enable (los `sdff*` son de scan y están excluidos de la síntesis en `no_synth.cells` del PDK), así que cada uno de los 480 bits del register file lleva un mux que elige entre mantener el valor o escribir `wdata`
`oai21_1`	439	0	51	15	373	Lógica combinacional general, sumadores y comparadores

Comparación con lo esperado

Antes de sintetizar se esperaba que el área estuviera dominada por los flip-flops, y dentro de ellos por el register file:

- **Register file.** Se esperaban 15 × 32 = 480 flip-flops de almacenamiento (`x0` cableado a 0) más 64 de los registros de salida de la lectura sincrónica: 544. Es exactamente lo que aparece. Lo que no se había previsto es la lógica que los rodea: los 480 muxes de habilitación de escritura y los dos árboles de lectura. Por eso los flip-flops son solo el 57 % del área del módulo, y el módulo llega al 43,7 % del total.
- **Contador de ciclos.** Con 32 de 1.098 flip-flops (2,9 %) se esperaba que fuera una fracción chica del área. Resultó 2,83 %, coherente: casi la mitad de su área son los flip-flops (51,6 % secuencial) y el resto, el incrementador.
- **ALU.** Es chica (6,1 %) porque los shifts están deshabilitados en Espino: solo quedan el sumador/restador, las comparaciones y las operaciones lógicas.
- **Memoria.** Con la estrategia ROM, el programa queda como lógica combinacional y la RAM de datos son 8 palabras de flip-flops sin reset (`dffq_1`). Ambas quedan dentro del "resto del SoC" (ver 3.2 para la comparación con la memoria de flip-flops).

Síntesis vs. layout final

Las áreas de arriba son de la síntesis. En el layout final cada bloque crece por los buffers que agrega el repair de OpenROAD (fanout, slew, capacitancia). La tabla de 3.6 (`docs/img/bloques.csv`, `scripts/capturas.py`) mide las mismas instancias en el layout final:

Bloque	Área en síntesis (µm²)	Área en el layout final (µm²)	Fracción en síntesis	Fracción del área lógica final
Register file	71.588	82.990	43,7 %	45,3 %
ALU	10.006	13.055	6,1 %	7,1 %
Contador de ciclos	4.632	4.983	2,83 %	2,7 %

Las dos mediciones coinciden en el orden y en la fracción del contador (2,7-2,8 %). En el layout, la memoria (ROM + RAM de datos, que en la síntesis queda dentro del "resto del SoC") se puede separar por los nombres de sus flip-flops: es el segundo bloque más grande (22,4 %).

3.4 Timing

Detalle completo, comandos y archivo:línea de cada número en `docs/timing.md`. Métricas en `results/timing.csv` (configuración final), `results/timing_con_dly.csv` (configuración anterior) y `results/optimizacion.csv` (L6).

Corners

LibreLane analiza 9 corners: parásitos `nom/min/max` × `tt_025C_5v00`, `ss_125C_4v50` y `ff_n40C_5v50`. En las tablas, **típico** = **`nom_tt_025C_5v00`** y **más lento** = **`max_ss_125C_4v50`** (proceso lento, 125 °C, 4,5 V y parásitos máximos). El peor hold aparece en los corners `ff` (0,13-0,14 ns en todas las corridas, siempre positivo).

Barrido de `CLOCK_PERIOD`

Configuración de la corrida final (memoria ROM, sin celdas de retardo, `FP_CORE_UTIL` = 40 %). WNS / TNS en ns. En todas las corridas, Magic DRC = 0, KLayout DRC = 0, LVS limpio y antena = 0.

T (ns)	Variante	Setup típico	Setup lento	Hold típico	Hold lento	Área (µm²)	¿Cierra?
100	por defecto (**corrida final**)	73,95 / 0	51,40 / 0	0,390 / 0	0,970 / 0	196.097	sí
60	por defecto	34,03 / 0	11,40 / 0	0,390 / 0	0,970 / 0	196.097	sí
50	por defecto	24,03 / 0	1,40 / 0	0,390 / 0	0,970 / 0	196.097	sí
40	por defecto	14,03 / 0	**−8,60 / −1.928,45**	0,390 / 0	0,970 / 0	196.097	**no**
40	margen de setup de 14 ns	20,58 / 0	3,55 / 0	0,390 / 0	0,970 / 0	197.320	sí
35	margen de setup de 14 ns	19,60 / 0	6,09 / 0	0,391 / 0	0,971 / 0	204.154	sí
30	margen de setup de 14 ns	14,70 / 0	1,28 / 0	0,390 / 0	0,971 / 0	203.960	sí

El menor período probado que cierra en todos los corners es 30 ns (33,3 MHz), con margen de setup en el resizer; con la configuración por defecto es **50 ns (20 MHz)**. No se probó nada por debajo de 30 ns, pero el límite parece cercano: de 35 a 30 ns el retardo de la ruta crítica casi no baja (27,91 → 27,71 ns).

Cada `CLOCK_PERIOD` re-optimiza el circuito, pero solo si el resizer de OpenROAD ve violaciones. Con la configuración por defecto, de 100 a 40 ns el circuito es el mismo (196.097 µm², 9.651 celdas) y el slack baja exactamente lo que baja el período. El resizer reporta `RSZ-0098 No setup violations found` incluso a 40 ns, donde el signoff encuentra 498 endpoints violando. El motivo es que optimiza con parásitos **estimados** desde el placement, que en el corner lento son mucho más optimistas que los **extraídos** del layout final. Medido a 60 ns con la configuración anterior: el resizer estimaba +9,38 ns, el signoff dio −3,54 ns.

`PL_RESIZER_SETUP_SLACK_MARGIN = 14` le exige 14 ns de sobra según su estimación, y ahí sí trabaja. Cuanto más corto el período, más cambia el circuito:

	40 ns con margen	35 ns con margen	30 ns con margen
Endpoints reparados	787	898	908
Celdas agrandadas	175	693	679
Buffers insertados	3	53	54
Área vs. corrida final	+0,6 %	+4,1 %	+4,0 %
Retardo de datos de la ruta crítica (`max_ss`)	35,28 ns	27,91 ns	27,71 ns

Además, LibreLane solo marca como error las violaciones de los corners típicos (`TIMING_VIOLATION_CORNERS = ['*tt*']`), así que el flujo termina "completo" aunque el corner lento viole. El criterio de cierre de las tablas se revisa en los 9 corners con `scripts/extract_metrics.py`.

Celdas de retardo

En las primeras corridas, la mitad del retardo de la ruta crítica eran celdas `dlyb_1`: el `repair_design` de OpenROAD las usaba como buffers de fanout (695 en todo el diseño), y cada una tarda 2,5 a 3,9 ns en el corner lento. La corrida final

las excluye (`EXTRA_EXCLUDED_CELLS`). Con el mismo RTL y la misma configuración, salvo esa opción (`CP100_U40_pines` vs. `CP100_U40_sindly`):

	Con `dly*`	Sin `dly*`
Setup WNS lento (T = 100 ns)	37,56 ns	54,23 ns
Retardo de datos de la ruta crítica	61,51 ns (64 % en `dly*`)	44,59 ns
Área de celdas	210.654 µm²	195.595 µm² (−7,1 %)
Hold peor	0,137 ns	0,134 ns

Con las `dly*`, el período mínimo era 70 ns por defecto y 60 ns con margen.

Ruta crítica de la corrida final

`CP100_U40_final`, corner `max_ss`, slack 51,40 ns:

Celda de inicio	**`_5393_` (`dffq_1`): bit 4 del registro de salida de la memoria de instrucciones, es decir, la instrucción ya leída (fin de la etapa de fetch)**
Celda de fin	`u_pochoco.u_periph.u_cycle_counter/_281_` (`dffrnq_1`): un bit del contador de ciclos
Módulos	memoria de instrucciones → decoder y mux de operandos (`espino_id_stage`) → `espino_alu` → decodificación de direcciones del SoC → `pochoco_periph` → `pochoco_cycle_counter`
Etapa de Espino	Espino tiene dos etapas: IF y una ID que en un ciclo hace decode, ejecución, acceso a memoria y write-back. Esta ruta recorre la ID desde el **decode** hasta el **acceso a memoria** (un store al contador)

Tramo	Desde → hasta	Retardo
Fetch	flanco de reloj → `a_rdata_o[4]`	2,01 ns
Decode	`opcode` → mux de `operand_b`	18,76 ns
Ejecución	ALU → `lsu_addr[31]`	15,26 ns
Acceso a memoria	bit 31 de la dirección → selección de periféricos → carga del contador	11,54 ns
Total		**47,56 ns**

Es la ruta de un `sw` al contador (offset `0x0C`): la ALU suma base + offset, y el **bit 31** de la dirección, el último que resuelve el sumador, decide si el acceso va a los periféricos. De ahí sale la señal de carga del contador. En el corner típico la peor ruta es de salida (registro → `seg2_o[6]`, slack 73,95 ns), porque el SDC le asigna a las salidas un retardo externo del 20 % del período.

Optimización de la ruta crítica

Con la configuración anterior, la ruta crítica iba de la instrucción al register file pasando por la **alineación de los loads**: el byte offset (`addr[1:0]`, que sale de la ALU) elegía qué byte del dato leído se escribía en el register file.

Cambio (`rtl/espino_core/espino_load_store_unit.v`): el offset se guarda en un registro de 2 bits (`boff_q`) en el ciclo en que se emite el load, y la alineación usa ese registro. Un load ya tardaba dos ciclos (la LSU detiene el core mientras la memoria lee) y la dirección no cambia entre ellos, así que **la latencia y el comportamiento no cambian**: solo se saca la ALU del camino alineación → write-back. Se verificó con un testbench nuevo (`sim/tb_load_store_unit.v`, 174 chequeos de todos los loads en todos los offsets; con una LSU saboteada fallan 24), y las simulaciones del juego dan idénticas byte a byte.

Resultado (T = 60 ns, configuración anterior, misma configuración en ambas):

	Antes	Con el cambio	Diferencia
Setup WNS lento	−3,54 ns	−2,63 ns	+0,91 ns
Setup TNS lento	−440,18 ns	−21,13 ns	−95 %
Endpoints que violan	191	10	−181
Área de celdas	211.325 µm²	211.391 µm²	+0,03 % (+2 FF)

La ruta atacada desaparece y las violaciones bajan de 191 a 10. El WNS mejora poco porque debajo había otra ruta casi igual de larga, la de los **branches**: decode → la ALU compara los operandos → `branch_taken` → mux de next-PC → registro del PC (etapa IF). Con la configuración final, sin celdas de retardo, la ruta crítica pasó a ser el store al contador

descrito arriba. El siguiente paso sería pre-decodificar `alu_op_b_sel` junto con la instrucción, para sacar el decode (el tramo más largo en todas estas rutas) del camino.

Constantes de `game.s` para el período T

El juego mide el tiempo en ciclos con el contador, así que cada constante de tiempo es `segundos × f`, con `f = 1 / T`. Se carga con `lui` (20 bits altos) + `addi` (12 bits bajos con signo). Si el bit 11 es 1, el `addi` queda negativo y el `lui` sube en 1.

Constante	**T = 100 ns, f = 10 MHz (corrida final)**	**T = 30 ns, f = 33,33 MHz (menor período probado que cierra)**
3 s	30.000.000 = `0x01C9C380` ⟶ `lui 0x1C9C` + `addi 896`	100.000.000 = `0x05F5E100` ⟶ `lui 0x5F5E` + `addi 256`
2 s	20.000.000 = `0x01312D00` ⟶ `lui 0x1313` + `addi -768`	66.666.667 = `0x03F940AB` ⟶ `lui 0x3F94` + `addi 171`
0,1 s	1.000.000 = `0x000F4240` ⟶ `lui 0xF4` + `addi 576`	3.333.333 = `0x0032DCD5` ⟶ `lui 0x32E` + `addi -811`

A 100 ns las constantes son exactas y son las de `sw/game.s`. A 30 ns, la de 3 s es exacta y las otras dos no son enteras y se redondean: el error es de 0,33 ciclos ($\leq 1 \times 10^{-7}$ relativo), despreciable frente a la décima de segundo que muestra el juego. Para usar otro período se cambian estas tres constantes en `sw/game.s` y se regenera el programa con `python3 -B sw/assembler.py sw/game.s sw/game.hex --rom rtl/game_rom.v`.

3.5 Floorplan, alimentación y placement

Todas las corridas usan el RTL final (memoria ROM, contador como módulo, `boff_q` en el LSU), `CLOCK_PERIOD` = 100 ns y el mismo `flow/asic_top/config.json`; solo cambia `FP_CORE_UTIL`.

```
scripts/run_sweep.sh -s final -p "100" -u "40 50 60 70 75 80"
python3 scripts/floorplan.py -o results/floorplan.csv -g results/floorplan.png \
  flow/asic_top/runs/CP100_U*_final
```

Floorplan

LibreLane dimensiona el core a partir del área de celdas que sale de la síntesis (`FP_SIZING = relative`): core = área de celdas / `FP_CORE_UTIL`, cuadrado (`FP_ASPECT_RATIO = 1`), y le suma un margen hasta el borde del die. Con U = 40 % el die mide 653,44 × 671,36 µm (0,4387 mm²). Después del floorplan se insertan las *tap cells* (cada `FP_TAPCELL_DIST` = 20 µm, conectan pozo y sustrato a la alimentación para evitar *latch-up*) y las *endcaps* en los extremos de cada fila.

La **utilización final** (48,0 % en la corrida final) es mayor que el `FP_CORE_UTIL` pedido, porque después del floorplan el flujo agrega buffers de repair, el árbol de reloj y la corrección de hold, y todo eso ocupa área dentro del mismo core.

Alimentación (PDN)

La red de alimentación la genera OpenROAD (`21-openroad-generatepdn`) con la configuración por defecto de LibreLane para GF180:

Nivel	Capa	Geometría
Rieles de las filas	Metal1	0,6 µm de ancho, uno por fila de celdas (VDD y VSS alternados)
Straps verticales	Metal4	1,6 µm de ancho, paso 153,6 µm
Straps horizontales	Metal5	1,6 µm de ancho, paso 153,18 µm

Los rieles de Metal1 alimentan a las celdas; los straps de Metal4/Metal5 forman una malla que lleva la corriente desde los terminales `VDD`/`VSS` del macro hasta los rieles, con vías en cada cruce. **No hay anillo de alimentación** (`PDN_CORE_RING = False`): el macro expone la malla como pines (`PDN_ENABLE_PINS`), y quien lo integre la conecta a la red del chip.

Verificación en la corrida final:

- 0 violaciones de la red de alimentación (`design__power_grid_violation__count`, etapa `30-checker-powergridviolations`): todos los segmentos de VDD y VSS están conectados (`PSM-0040`).
- IR drop estático en `nom_tt` (`56-openroad-irdropreport/irdrop.rpt`): peor caída 0,143 mV en VDD y 0,097 mV en VSS, con una alimentación de 5 V. Es despreciable porque el diseño consume poco: 13,0 mW en total según `final/metrics.json` (`power__total`), de los cuales 10,1 mW son internos, 2,9 mW de conmutación y 2,4 µW de fuga.

Terminales del macro

`flow/asic_top/pin_order.cfg` agrupa los terminales por función, un lado por grupo. Con `ERRORS_ON_UNMATCHED_IO = both`, el flujo se detiene si falta un puerto o si sobra uno.

Lado	Terminales	Por qué ahí
Oeste	`rst_ni`, `clk_i`	Señales globales con un lado propio: la raíz del árbol de reloj parte del borde sin competir con buses de datos, y el reset entra junto al reloj al sincronizador
Sur	`btn_i[3:0]`, `led_o[3:0]`	Interfaz de usuario: ambos buses terminan en `pochoco_periph` (debouncer y registro de LEDs)
Norte	`seg1_o[6:0]`, `seg2_o[6:0]`	El bus más ancho (14 salidas), cada display con sus 7 segmentos contiguos y en orden, para rutearlos sin cruces al integrar el macro
Este	`spi_cs_ni`, `spi_sclk_i`, `spi_mosi_i`, `spi_miso_o`	Interfaz independiente (`pochoco_spi_slave`)

Los terminales son rectángulos de metal en el borde del macro (Metal2 en norte y sur, Metal3 en este y oeste), sin ningún circuito propio: los maneja directamente una celda estándar interna. **Un chip completo usaría pads de I/O**: cada señal pasaría por una celda de pad (`gf180mcu_fd_io`) con protección ESD, un driver capaz de cargar una pista de PCB y un *bondpad* para el cable de conexión, más pads dedicados de alimentación, todos en un anillo alrededor del core. En un

chip el orden de los pads lo fija el encapsulado y la placa. En un macro, en cambio, se elige para facilitar el ruteo hacia el resto del chip.

Barrido de utilización

FP_CORE_UTIL	Die (µm)	Die (mm²)	Util. final	Uso ruteo global	Viol. DRT iter. 0	Setup WS ss (ns)	Hold WS peor (ns)	¿Cierra?	Costo (US$)
40 %	653,4 × 671,4	0,4387	48,0 %	23,34 %	51	51,40	0,143	sí	10.090
50 %	585,9 × 603,8	0,3537	59,0 %	26,91 %	3.146	53,37	0,120	sí	8.136
60 %	536,0 × 553,9	0,2969	70,2 %	30,54 %	56	52,89	0,138	sí	6.829
70 %	497,2 × 515,2	0,2562	81,6 %	34,97 %	57	51,88	0,120	sí	5.892
75 %	**480,8 × 498,8**	**0,2398**	86,9 %	39,06 %	2.742	51,25	0,124	**sí**	**5.516**
80 %	—	—	—	—	—	—	—	no: el flujo no termina	—

Fuente: `results/floorplan.csv`. En todas las que cierran: DRC Magic y KLayout 0, LVS 0, antena 0, setup y hold con WS ≥ 0 y TNS = 0 en los 9 corners.

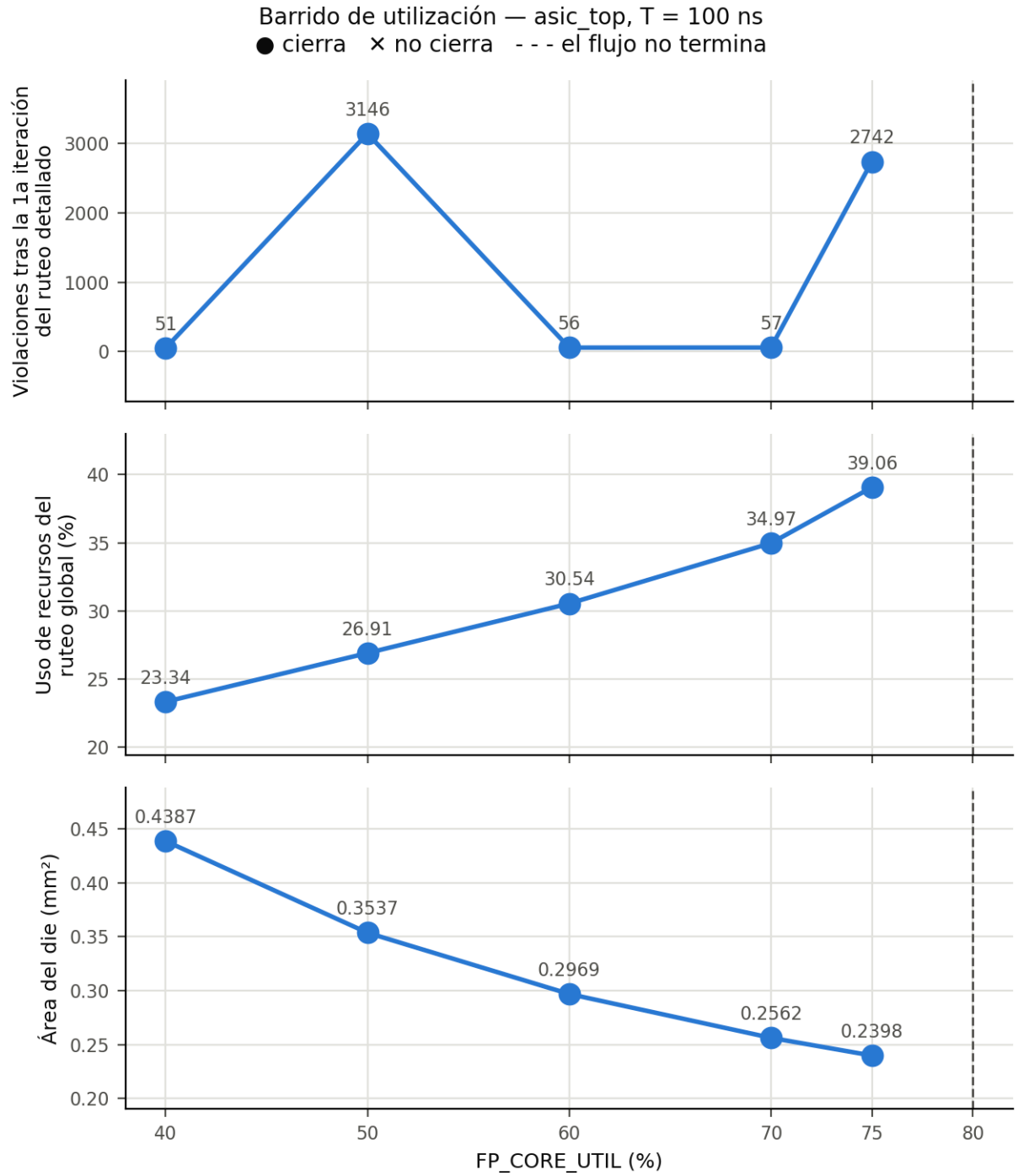


Figura 1: Barrido de utilización

Área mínima que cierra: `FP_CORE_UTIL` = 75 %, die de 0,2398 mm² (480,8 × 498,8 µm). A 80 % el flujo se detiene en CTS, antes de rutear: `DPL-0034`, 10 instancias que no se pueden legalizar (3 del register file), y `DPL-0036 Detailed placement failed` (`CP100_U80_final/35-openroad-cts/openroad-cts.log:109` y `:120`).

Análisis.

- **Congestión.** El uso de recursos del ruteo global crece de forma monótona con la utilización, de 23 % a 39 %: hay más cables en menos área. Es la métrica de congestión más limpia del barrido.
- **Violaciones del ruteo detallado.** Las que quedan tras la primera iteración **no** siguen una tendencia (51, 3.146, 56, 57, 2.742): dependen de cómo quedó el placement en cada punto, y el ruteador las lleva a 0 en todas las corridas que terminan. Muestran el trabajo extra del ruteador, no un límite.
- **Timing.** Casi no cambia: el setup en ss está entre 51 y 53 ns en todos los puntos, porque a 100 ns sobra holgura.

- **Qué limita.** El **espacio para legalizar celdas**. A 80 % el placement inicial cabe, pero las celdas que se agregan después (buffers del árbol de reloj) ya no encuentran lugar libre en las filas.

En un barrido anterior, con las celdas de retardo `dly*` todavía permitidas, 75 % también fallaba en CTS. Lo más probable es que fuera por el tamaño de esas celdas: `repair_design` usaba `dlyb_1` como buffer, y ocupa 2,7 veces el área de un `buf_1` (detalle en `docs/floorplan.md`).

Costo

A US$23.000/mm², escalando linealmente con el área del die:

- Corrida exigida (U = 40 %): 0,4387 mm² ⟶ **US$10.090**.
- Área mínima que cierra (U = 75 %): 0,2398 mm² ⟶ **US$5.516** (−45 %).

Limitaciones de la aproximación:

- **Es solo el macro.** Un chip real necesita el anillo de pads, que en un diseño tan chico ocuparía más área que el core: el die quedaría limitado por los pads y no por la lógica.
- **El costo no es lineal.** Hay costos fijos (máscaras, diseño, verificación) que no dependen del área, y en un shuttle multiproyecto se compra un espacio de tamaño mínimo.
- **No incluye rendimiento** (la probabilidad de un defecto crece con el área), ni corte del wafer, encapsulado o test.
- **Vale solo para GF180MCU**; no sirve para extrapolar a otro nodo.

Decisión que más pesó en el área y el costo: dimensionar la memoria al uso real del juego. Con la RAM de 512 palabras del Proyecto 2, la síntesis ya daba 2,31 mm² solo de celdas (`flow/asic_top/runs/01_regfile_ff`, sección 3.1). Con la ROM, la corrida final tiene 0,196 mm² de celdas, unas 12 veces menos. Entre las dos estrategias de 3.2, comparadas con el mismo RTL y la misma configuración (`results/memoria.csv`), la de flip-flops da un die de 1,571 mm² (US$36.133) y la ROM uno de 0,4407 mm² (US$10.137): 3,6 veces más. La utilización viene después y solo mueve el die dentro de un factor ~1,8 (0,439 ⟶ 0,240 mm²).

3.6 Signoff y layout

Corrida final `flow/asic_top/runs/CP100_U40_final` (T = 100 ns, U = 40 %, memoria ROM). Detalle completo y rutas de cada reporte en `docs/signoff.md`.

Resultado del signoff

Chequeo	Resultado	Reporte (en la carpeta de la corrida)
DRC Magic	**0**	`64-magic-drc/reports/drc.magic.rpt`
DRC KLayout (558 reglas del PDK)	**0**	`65-klayout-drc/reports/drc.klayout.json`
XOR entre el GDS de Magic y el de KLayout	**0** diferencias	`final/metrics.json`
LVS (Netgen)	**"Circuits match uniquely"**	`70-netgen-lvs/reports/lvs.netgen.rpt`
Antena (final)	**0** redes, 0 pines	`46-openroad-checkantennas-1/reports/antenna_summary.rpt`
Setup / hold (9 corners)	WS ≥ 0, TNS = 0	`55-openroad-stapostpnr/summary.rpt`

Dos DRC independientes. Con `gf180mcuD`, LibreLane trae `KLAYOUT_DRC_RUNSET` sin definir y salta el DRC de KLayout; en las primeras corridas, el "DRC Passed" venía solo de Magic. Se habilitó el runset del PDK (`libs.tech/klayout/tech/drc/gf180mcu.drc`, con la variante de metalización de `gf180mcuD`: 5 metales, metal superior de 11 kÅ, MIM opción B). En la corrida final ejecuta 558 reglas (FEOL, BEOL, conectividad y *off-grid*) sin violaciones, y coincide con Magic sobre el mismo GDS.

LVS compara el netlist extraído del layout con el netlist de PnR. Coinciden 1 a 1, incluidos los 3 diodos de antena que agregó el repair.

Violación de antena durante el desarrollo

En la corrida final: 4 violaciones, corregidas por el flujo

Después del ruteo global, `40-openroad-checkantennas` encontró **4 redes que violan la regla de antena** (razón de área lateral máxima 400):

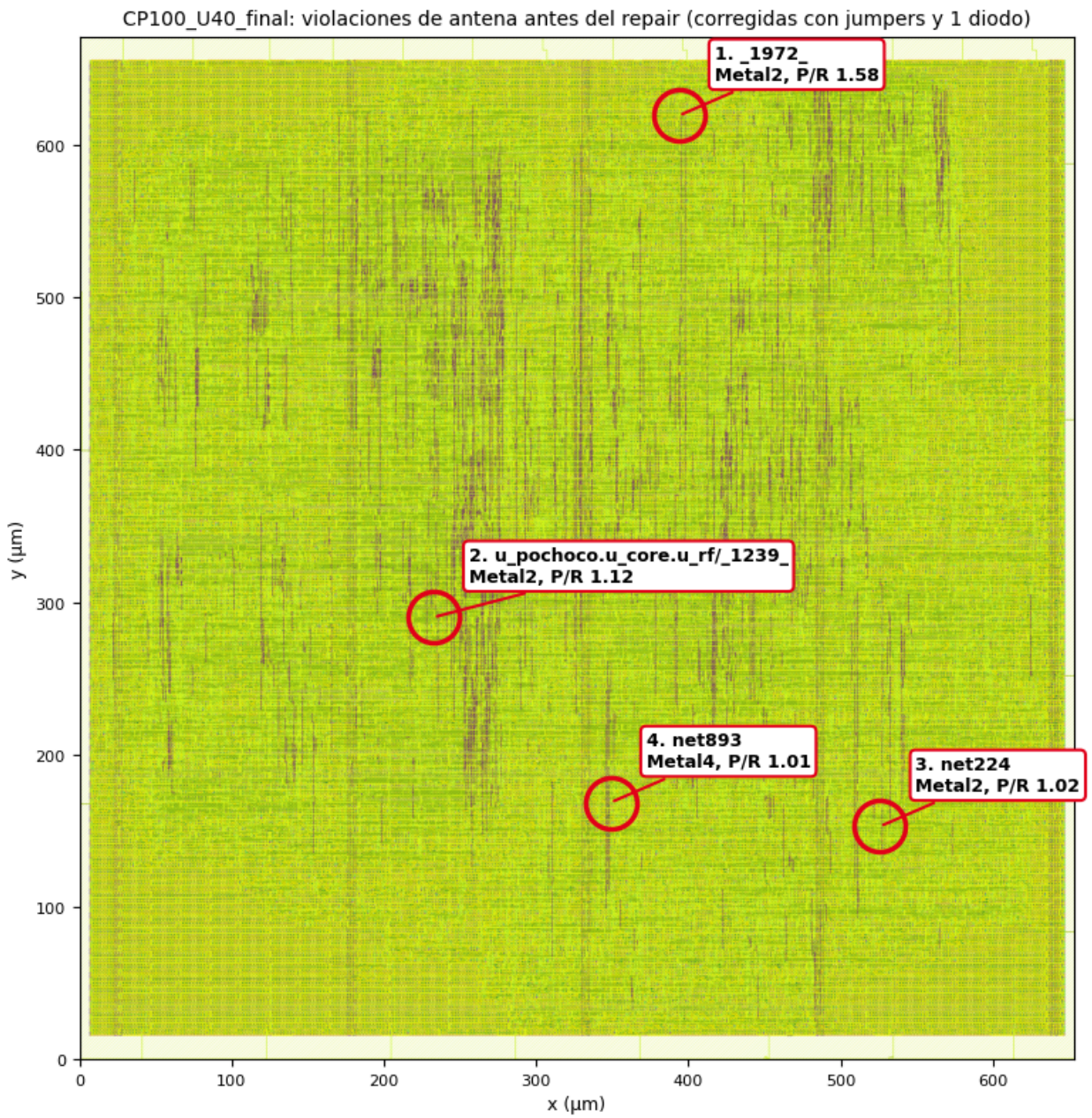


Figura 2: Ubicación de las violaciones de antena

#	Red	Pin	Capa	Razón / máx.	Posición (µm)	Zona del layout
1	`_1972_`	`fanout29/I`	Metal2	633,66 / 400	(393,7; 619,4)	borde norte, entre el register file y la memoria
2	`u_pochoco.u_core.u_rf/_1239_`	`fanout296/I`	Metal2	446,64 / 400	(232,4; 290,1)	dentro del register file (mux de lectura)
3	`net224`	`fanout223/I`	Metal2	406,35 / 400	(525,8; 152,9)	zona sur de la memoria
4	`net893`	`fanout878/I`	Metal4	402,02 / 400	(348,9; 168,6)	red de reset, debajo del contador

Fuente: `40-openroad-checkantennas/reports/antenna_summary.rpt`; posiciones de las celdas en `final/def/asic_top.def`. Figura: `python3 scripts/antenas_layout.py`.

Qué es. Mientras se fabrican las capas de metal, el grabado por plasma deposita carga en los conductores. Si un tramo de metal está conectado solo a la compuerta de un transistor (todavía no hay difusión que lo descargue) y su área es mucho mayor que la de esa compuerta, el voltaje acumulado puede romper el óxido de compuerta. Las 4 redes son tramos largos que terminan en la entrada de un `buf_1`, que tiene una compuerta chica.

Cómo se resolvió (`42-openroad-repairantennas`, log de `1-openroad-diodeinsertion`):

1. 4 violaciones → **3 jumpers** en 3 redes. Un jumper parte el tramo largo con un puente a una capa superior: mientras se fabrican las capas inferiores, la compuerta queda conectada a un tramo más corto. Queda 1 violación.
2. 1 violación → **1 diodo de antena** junto al pin, que descarga la carga a sustrato. Al reacomodar aparecen 2 violaciones nuevas.
3. 2 violaciones → **2 jumpers más** → 0.

El chequeo final, después del ruteo detallado (`46-openroad-checkantennas-1`), da 0.

En la memoria de flip-flops: 1 violación que no se pudo corregir

La corrida de la estrategia 2 (`flow/asic_top_ff/runs/CP100_U40_ff`, sección 3.2) es un caso donde la corrección automática no alcanzó:

- Tras el ruteo global había **125 redes** con violación. El repair las bajó a 0 con 157 jumpers y 145 diodos (159 → 108 → 91 → 12 → 5 → 3 → 0).
- **El ruteo detallado posterior volvió a crear 1**: `net4125` → `fanout4124/I`, Metal2, 472,61 / 400, en (968,2; 109,8) µm, cerca del borde sur (`46-openroad-checkantennas-1/reports/antenna_summary.rpt`).

La causa es que el repair de antenas trabaja sobre el ruteo **global**. El ruteo detallado puede cambiar el recorrido de una red, y en un diseño tan congestionado cambió el de esta y dejó otra vez un tramo largo de Metal2. Por esa sola violación, la estrategia de flip-flops **no cierra** con el criterio del enunciado. No se corrigió porque la estrategia elegida fue la ROM. Las opciones habrían sido subir `GRT_ANTENNA_REPAIR_MARGIN` para que el repair deje más holgura, o insertar un diodo en ese pin.

Violaciones fuera del criterio: max slew, cap y fanout

LibreLane también reporta transiciones mayores a 3 ns (**max slew**), capacitancia y fanout excesivos. No son parte del criterio de cierre y LibreLane no las trata como error, pero se analizaron:

Corner	Max slew	Max cap
`nom_tt_025C_5v00`	21	4
`max_ss_125C_4v50`	2.867	10

`repair_design` dimensiona los buffers en el corner típico, dejando 20 % de margen sobre 3 ns. En el corner lento las celdas son más lentas y esas transiciones superan el límite. En `max_ss`, la mayoría está en el register file (1.435 pines, los selects de sus mux de lectura de 15 entradas) y en el resto del core (1.083). El peor pin (7,08 ns) está en la **red de reset**, la misma red `net893` que tuvo la violación de antena #4: un árbol de `buf_1` que reparte el reset a todos los flip-flops (`scripts/slew_por_bloque.py`).

Aun así, el setup tiene 51 ns de slack en `max_ss`, y la liberación del reset está sincronizada y verificada por los chequeos de recovery/removal. Se intentó subir el margen del repair a 60 % (`DESIGN_REPAIR_MAX_SLEW_PCT`), pero en dos corridas la etapa de repair no terminó (sin ese cambio tarda menos de un minuto), así que se descartó.

Layout anotado

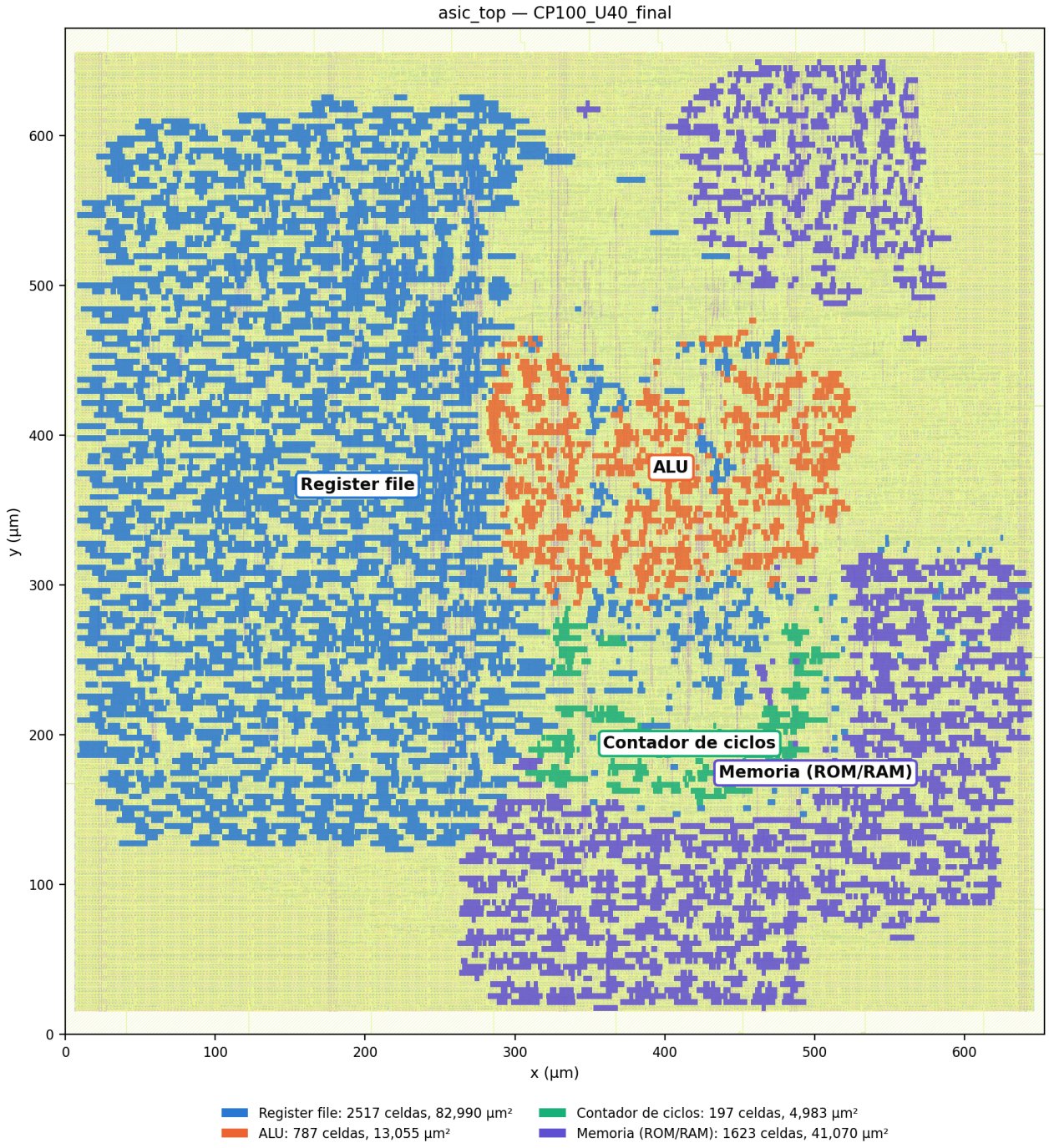


Figura 3: Layout final con bloques marcados

Render del GDS final con KLayout (`docs/img/layout.png`) y las celdas de cada bloque coloreadas (`docs/img/layout_anotado.png`, `docs/img/bloques.png`), generado con `python3 scripts/capturas.py flow/asic_top/runs/CP100_U40_final -o docs/img`.

Bloque	Instancia	Celdas	FF	Área (µm²)	% área lógica
Register file	`u_pochoco.u_core.u_rf`	2.517	544	82.990	45,3 %
Memoria (ROM + datos)	`u_pochoco.g_mem_rom.u_ram`	1.623	319	41.070	22,4 %
ALU	`u_pochoco.u_core.u_alu`	787	0	13.055	7,1 %

Bloque	Instancia	Celdas	FF	Área (µm²)	% área lógica
Contador de ciclos	u_pochoco.u_periph.u_cycle_counter	197	32	4.983	2,7 %

Cómo se identificó cada bloque. El netlist final está aplanado, pero `SYNTH_KEEP_HIERARCHY_MODULES` hace que las celdas del register file, la ALU y el contador conserven el prefijo de su instancia (`u_rf/…`, `u_alu/…`, `u_cycle_counter/…`). Para la memoria, que no está en esa lista, se usa el nombre de la red de salida de cada flip-flop, que Yosys conserva desde el RTL, más la lógica combinacional que alimenta solo a esos flip-flops.

Qué se ve. El register file es el bloque más grande y ocupa casi toda la mitad oeste. La ALU queda en el centro, junto al register file, que le entrega los operandos. El contador está debajo de la ALU. La memoria se reparte en dos zonas, noreste y sur. Como `PL_TIMING_DRIVEN` está desactivado, el placer agrupa las celdas por conectividad (longitud de cable) y no por timing.

3.7 Simulación funcional post-síntesis

El objetivo es mostrar que el netlist que genera LibreLane se comporta igual que el RTL: mismo programa, mismos estímulos, mismos ciclos. Detalle y comandos en `docs/simulacion_post_sintesis.md`.

Programa con los tiempos acortados

A 10 MHz, la espera de 3 s son 30 millones de ciclos por ronda, y simular eso sobre el netlist toma horas. `scripts/game_sim.py` genera `sim/build/game_sim.s` desde `sw/game.s` dividiendo por 100 las constantes de tiempo:

Constante	`game.s` (10 MHz)	`game_sim.s`
Espera antes del objetivo (3 s)	30.000.000 ciclos	300.000
Pausa (2 s)	20.000.000	200.000
Décima de segundo	1.000.000	10.000

Las instrucciones y las direcciones no cambian, solo esos inmediatos (recalculados como `lui + addi`). El hardware tampoco cambia: el *debounce* sigue con `TICK_BITS = 17` (muestrea cada 131.072 ciclos y acepta un cambio después de 2 muestras iguales), así que el testbench mantiene cada botón apretado 300.000 ciclos. **Las dos simulaciones, RTL y netlist, usan este mismo programa y esta misma configuración.**

Testbench

`sim/tb_game_fixed_stimuli.v` instancia `asic_top`, el mismo top que sintetiza LibreLane, así que el mismo archivo sirve para el RTL y para el netlist. Aprieta los botones en **ciclos fijos**, sin mirar los LEDs para decidir. Si el netlist tuviera un ciclo de diferencia en cualquier punto, los eventos quedarían corridos y la comparación fallaría. El juego es determinista desde el reset (el generador pseudoaleatorio se siembra con el contador de ciclos), así que el objetivo de la primera ronda es siempre `0010`.

Escenario:

1. Ciclo 350.000: botón `0001`, con el objetivo `0010` encendido (botón incorrecto).
2. Ciclo 1.000.000: botón `0010`, en la ronda reiniciada (botón correcto).

El testbench registra cada cambio de los LEDs y de los displays con el ciclo en que ocurre, y al final revisa que haya aparecido `EE`, que la ronda se haya reiniciado y que se haya medido una ronda correcta.

Resultado en RTL

Log de `sim/run_fixed_stimuli.sh rtl`:

Ciclo	LEDs	Display	Qué pasó
21	`1111`	`00`	Sale del reset: fase inicial, los 4 LEDs encendidos
300.045	`0010`	`00`	Termina la espera (300.000 ciclos): se enciende el objetivo
589.848	`0000`	`EE`	El *debounce* acepta el botón incorrecto: error
639.954	`1111`	`00`	**La ronda se reinicia**
939.978	`0010`	`00`	Nueva espera de 300.000 ciclos: objetivo encendido
1.245.563	`0010`	`30`	El *debounce* acepta el botón correcto: se muestra el tiempo

La ronda correcta mide **305.585 ciclos**, desde que se enciende el objetivo hasta que se muestra el resultado. Con una décima de 10.000 ciclos son 30,5 décimas, y el display muestra `30` (el programa trunca). Ese tiempo incluye los ~245.000 ciclos que tarda el *debounce* en aceptar la pulsación, que en la FPGA también están (a 10 MHz son ~25 ms).

El testbench además cuenta los accesos de datos a RAM (0) y la dirección de instrucción más alta pedida (`0x18c`), que respaldan el conteo de palabras de 3.2.

Resultado en el netlist

El netlist incluye el programa: con la ROM, cada palabra se convierte en compuertas. Por eso se sintetiza aparte, con la ROM de `game_sim.s`, en `flow/asic_top_sim`. Ese flujo tiene la **misma configuración de síntesis** que la corrida final (`flow/asic_top`): los mismos archivos RTL, `SYNTH_KEEP_HIERARCHY_MODULES` con el register file, la ALU y el contador, y las

celdas `dly*` excluidas. Las únicas diferencias son el contenido de la ROM y los parámetros de pasos posteriores a la síntesis (pines, DRC de KLayout), que no afectan el netlist.

El netlist (`CP100_U40_sim2/06-yosys-synthesis/asic_top.nl.v`, 5.617 celdas) se simula con Icarus usando los modelos Verilog de las celdas de GF180 (`gf180mcu_fd_sc_mcu7t5v0`) en modo funcional, sin retardos. Después, `run_fixed_stimuli.sh compara` revisa los dos logs línea por línea:

```
IGUALES: sim/build/rtl_rom.log y sim/build/netlist.log tienen los mismos eventos en los mismos ciclos
```

- **Ronda correcta:** el netlist mide los mismos **305.585 ciclos** y muestra `30`.
- **Pulsación incorrecta:** el netlist muestra `EE` en el ciclo 589.848 y reinicia la ronda en el ciclo 639.954, igual que el RTL.

Se hizo dos veces: primero con una síntesis anterior a la optimización del LSU de 3.4 (`CP100_U40_sim`, 67 min de simulación) y después con el RTL final (`CP100_U40_sim2`). Las dos dieron **IGUALES**.

Qué demuestra y qué no

- **Demuestra** que la síntesis conserva la función del diseño completo, incluidos la ROM generada por el assembler, el *debounce*, el contador de ciclos y el core.
- **Da evidencia de que el reset alcanza.** En el netlist no hay bloques `initial` ni valores iniciales: todos los FF arrancan en `X`. Los que tienen reset toman un valor conocido con el reset externo (3.1). Los 319 sin reset (`dffq_1`), entre ellos la RAM de datos que el juego no usa y los registros de salida de la memoria, no llevan `X` a las salidas durante la simulación: los eventos coinciden con los del RTL. Si el juego dependiera de un valor inicial (como el `initial` del *debounce* del Proyecto 2), esas `X` podrían llegar a las salidas y la comparación lo mostraría. Es evidencia a nivel lógico, no una prueba del comportamiento en silicio.
- **No demuestra** que el diseño cumpla timing: la simulación es sin retardos. Eso lo garantiza el análisis estático de 3.4 y 3.6 (setup y hold con WS ≥ 0 y TNS = 0 en todos los corners). La simulación post-PnR con SDF, que es opcional, no se hizo.
- Los tiempos acortados no cambian el control del programa: solo cambian inmediatos, y las dos simulaciones usan el mismo programa.

Cómo correrlo

```
sim/run_fixed_stimuli.sh rtl                # RTL, ~1,5 min -> sim/build/rtl_rom.log
scripts/run_sweep.sh -d flow/asic_top_sim -s sim2 -p "100" -u "40" -- --to Yosys.Synthesis
sim/run_fixed_stimuli.sh netlist flow/asic_top_sim/runs/CP100_U40_sim2/06-yosys-synthesis/asic_top.nl.v
sim/run_fixed_stimuli.sh compara sim/build/rtl_rom.log sim/build/netlist.log
```